M02 - SQL Foundations

14-lesson beginner-first teaching book - RC2

How to use this book

For each lesson: understand -> follow the worked example -> check the expected result -> explain why it worked -> do guided practice -> close the answer -> attempt the independent task. You are never expected to infer hidden instructions.

Contents

ID	Lesson
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-01' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-01')], rise=0, text='DE02-01', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Database, table, row, column and query mental model' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-01')], rise=0, text='Database, table, row, column and query mental model', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-02' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-02')], rise=0, text='DE02-02', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'SELECT, aliases and LIMIT' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-02')], rise=0, text='SELECT, aliases and LIMIT', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph

ID	Lesson
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-03' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-03')], rise=0, text='DE02-03', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'WHERE and comparison operators' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-03')], rise=0, text='WHERE and comparison operators', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-04' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-04')], rise=0, text='DE02-04', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'AND, OR, NOT and parentheses' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-04')], rise=0, text='AND, OR, NOT and parentheses', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph

ID	Lesson
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-05' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-05')], rise=0, text='DE02-05', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'ORDER BY, DISTINCT and result inspection' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-05')], rise=0, text='ORDER BY, DISTINCT and result inspection', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-06' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-06')], rise=0, text='DE02-06', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'NULL, IS NULL and COALESCE' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-06')], rise=0, text='NULL, IS NULL and COALESCE', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph

ID	Lesson
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-07' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-07')], rise=0, text='DE02-07', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Arithmetic expressions and safe calculations' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-07')], rise=0, text='Arithmetic expressions and safe calculations', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-08' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-08')], rise=0, text='DE02-08', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Aggregate functions' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-08')], rise=0, text='Aggregate functions', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph

ID	Lesson
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-09' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-09')], rise=0, text='DE02-09', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'GROUP BY and grouped grain' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-09')], rise=0, text='GROUP BY and grouped grain', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-10' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-10')], rise=0, text='DE02-10', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'HAVING after aggregation' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-10')], rise=0, text='HAVING after aggregation', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph

ID	Lesson
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-11' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-11')], rise=0, text='DE02-11', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'CASE for business categories' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-11')], rise=0, text='CASE for business categories', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-12' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-12')], rise=0, text='DE02-12', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'String and date basics' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-12')], rise=0, text='String and date basics', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph

ID	Lesson
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-13' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-13')], rise=0, text='DE02-13', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Simple subqueries and IN/EXISTS awareness' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-13')], rise=0, text='Simple subqueries and IN/EXISTS awareness', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'DE02-14' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-14')], rise=0, text='DE02-14', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Debugging and validation workflow' 'frags': [ParaFrag(__tag__='u', bold=0, fontName='Helvetica', fontSize=9.6, greek=0, italic=0, link=[(0, '#DE02-14')], rise=0, text='Debugging and validation workflow', textColor=Color(.090196,.196078,.301961,1), us_lines=[(0, 'underline', None, '', '-0.125*F', 0, 1, '1')])] 'style': 'bulletText': None 'debug': 0) #Paragraph

Optional class video: Alex The Analyst SQL Beginner to Advanced

DE02-01 - Database, table, row, column and query mental model

What you are learning

Understand what you are looking at before writing SQL: database, table, row, column, query editor and result grid.

Why this matters

If these pieces are blurry, every later command feels like random code. Once the mental model is clear, SQL becomes a way to ask a table a precise question.

Picture it in your head

Think of a database as a well-organized office cabinet. A table is one sheet inside it. A row is one record, a column is one kind of fact, and a query is the question you ask without rewriting the original sheet.

Start from zero

A **database** is a container for related data objects. In this module our database is **stage02_retail_lab**.

A **table** stores data in rows and columns. Our main table is **sales_orders**.

A **row** is one record. In sales_orders, one row represents one order.

A **column** is one property of every row: order_id, order_date, region, qty and so on.

A **query** is a statement that asks MySQL to return or calculate something. A SELECT query normally reads; it does not rewrite the table.

The **result grid** is a temporary answer. Seeing five rows in the result does not mean the table now contains only five rows.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
SELECT order_id, region, status
FROM sales_orders
LIMIT 5;
```

Read the query in plain English

- **SELECT** says which columns you want to see.
- **FROM sales_orders** says where those columns come from.
- **LIMIT 5** asks MySQL to display only five rows for inspection.
- The semicolon **;** marks the end of the statement.

What success looks like

order_id	region	status
O1001	East	Completed
O1002	West	Completed
O1003	North	Completed
O1004	South	Completed

order_id	region	status
O1005	East	Completed

Common beginner mistakes

- Believing the result grid is the stored table itself.
- Running code while the wrong connection/database is selected.
- Trying to memorize keywords before understanding what each line is asking for.

Now you do it

Without copying, return order_id, order_date and product for five rows. Then say aloud: "One stored row is one order; my result is only a view of selected columns."

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents

DE02-02 - SELECT, aliases and LIMIT

What you are learning

Choose the columns you need, give readable names to calculated outputs, and inspect a small sample safely.

Why this matters

Real tables can have dozens or hundreds of columns. Good analysts request only what the question needs and label calculations so another person can understand the output.

Picture it in your head

SELECT is like telling a waiter exactly which dishes to bring. An alias is the label on the plate. LIMIT is asking for a small tasting sample, not claiming that the whole kitchen contains only five dishes.

Start from zero

Write column names after **SELECT**, separated by commas.

Use **AS** to give a result column a temporary readable name. This does not rename the stored table column.

LIMIT is excellent for inspection while learning or checking a huge table.

Never use a limited sample to report a business total unless the question explicitly asks for a sample.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
SELECT order_id,
       qty,
       unit_price,
       qty * unit_price AS gross_sales
FROM sales_orders
LIMIT 10;
```

Read the query in plain English

- qty * unit_price is calculated for each surviving row.
- AS gross_sales names the calculated result column.
- The stored sales_orders table is unchanged.

What success looks like

order_id	qty	unit_price	gross_sales
O1001	2	850.00	1700.00
O1002	3	450.00	1350.00
...	...	...	...

Common beginner mistakes

- Using SELECT * forever instead of learning which columns matter.
- Thinking AS permanently renames a database column.
- Reporting the first 10 rows as if they represent the full table.

Now you do it

Return order_id, customer_id, product and a calculated gross_sales column for 7 rows. Click the result header and confirm the alias is visible.

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents

DE02-03 - WHERE and comparison operators

What you are learning

Filter rows with WHERE using comparison operators and quoted text values.

Why this matters

Most business questions are not "show me everything". They ask about a subset: completed orders, dates in a range, large quantities, one region, and so on.

Picture it in your head

WHERE is a security gate. Every stored row approaches the gate; only rows whose condition is TRUE are allowed into the result.

Start from zero

= means equal, **<>** means not equal, and >, >=, <, <= compare values.

Text values such as **'Completed'** need quotes. Numeric values such as 2 normally do not.

WHERE does not delete the rows that fail the condition. It only leaves them out of this result.

Validate a filter by counting the returned rows and spot-checking a few records.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
SELECT order_id, region, qty, status
FROM sales_orders
WHERE status = 'Completed'
  AND qty >= 2;
```

Read the query in plain English

- status = 'Completed' keeps completed orders.
- qty >= 2 keeps orders with quantity 2 or more.
- Both conditions must be true because AND is used.

What success looks like

check	expected
Returned rows	22
Cancelled/Pending rows in result	0

Common beginner mistakes

- Writing status = Completed without quotes.
- Using = when the business rule says at least (>=).
- Assuming a plausible row count means the filter is definitely correct.

Now you do it

Return orders dated from 2026-07-10 through 2026-07-12 inclusive. Record the result count before looking at any review.

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents

DE02-04 - AND, OR, NOT and parentheses

What you are learning

Translate English business rules into AND, OR, NOT and explicit parentheses.

Why this matters

Many wrong SQL answers are syntactically valid but logically wrong because AND and OR were grouped differently from the business sentence.

Picture it in your head

Imagine brackets in mathematics. Parentheses tell MySQL which pieces of the rule belong together before it combines them with another condition.

Start from zero

AND: both sides must be true, so it usually narrows the result.

OR: either side may be true, so it usually broadens the result.

NOT reverses a condition. Often a direct comparison such as status <> 'Cancelled' is easier to read.

Use parentheses even when you know operator precedence. The goal is readable, auditable business logic.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
SELECT order_id, region, status
FROM sales_orders
WHERE status = 'Completed'
  AND (region = 'East' OR region = 'North');
```

Read the query in plain English

- The parentheses create one region rule: East OR North.
- AND then requires the order to be Completed as well.
- In the supplied data this rule returns 13 rows.

What success looks like

validation	expected
Rows returned	13
Allowed regions	East, North
Allowed status	Completed only

Common beginner mistakes

- Writing status=Completed AND region=East OR region=North without parentheses and accidentally allowing North rows of another status.
- Using OR where the English sentence really says all conditions must hold.

Now you do it

Return orders that are not Cancelled and whose payment method is Card or UPI. Write the English rule above your SQL as a comment first.

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents

DE02-05 - ORDER BY, DISTINCT and result inspection

What you are learning

Sort result rows, remove duplicate result combinations when appropriate, and inspect outputs deliberately.

Why this matters

"Top", "highest", "latest" and "unique list" questions require ORDER BY or DISTINCT. Without them, the result can look convincing while answering a different question.

Picture it in your head

ORDER BY rearranges the cards on your desk; it does not rewrite the filing cabinet. DISTINCT is asking for one copy of each unique card label in the result.

Start from zero

ORDER BY column ASC sorts smallest-to-largest / A-to-Z. ASC is the default.

DESC reverses the order.

DISTINCT removes duplicate combinations from the selected result columns.

Do not sprinkle DISTINCT over a query to hide duplicate rows caused by wrong join logic. In this module we use it mainly for legitimate unique lists.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
SELECT DISTINCT region
FROM sales_orders
ORDER BY region;
```

Read the query in plain English

- Only region is selected, so DISTINCT produces one row per unique region value.
- ORDER BY region sorts the four labels alphabetically.

What success looks like

region
East
North
South
West

Common beginner mistakes

- Assuming stored table order is meaningful.
- Using LIMIT 5 without ORDER BY when the question asks for the top five.
- Using DISTINCT to hide unexplained duplicates.

Now you do it

Return the five orders with the highest gross_sales. Use a calculated alias, sort descending, then LIMIT 5. The highest should be O1020 with 3600.

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents

DE02-06 - NULL, IS NULL and COALESCE

What you are learning

Handle missing values correctly with IS NULL, IS NOT NULL and COALESCE.

Why this matters

NULL does not mean zero, empty text or "No". It means the value is missing/unknown/not recorded for this column. SQL treats NULL differently from normal values.

Picture it in your head

A blank answer box on a form is not the number 0 and not the word "none". It is simply not filled in. SQL uses NULL to represent that absence.

Start from zero

Do not test missing values with **= NULL**. Use **IS NULL** or **IS NOT NULL**.

COUNT(*) counts rows. **COUNT(sales_rep)** counts only non-NULL sales_rep values.

COALESCE(sales_rep, 'Unassigned') can display a replacement in the result without editing the stored data.

Always ask whether replacing NULL is meaningful. Sometimes leaving NULL visible is safer.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
SELECT COUNT(*) AS all_rows,
       COUNT(sales_rep) AS rows_with_rep,
       COUNT(*) - COUNT(sales_rep) AS missing_rep
FROM sales_orders;
```

Read the query in plain English

- There are 30 rows total.
- There are 29 non-NULL sales_rep values.
- Therefore one order has no sales_rep recorded.

What success looks like

all_rows	rows_with_rep	missing_rep
30	29	1

Common beginner mistakes

- WHERE sales_rep = NULL.
- Treating NULL as an empty string automatically.
- Replacing NULL without documenting the business meaning.

Now you do it

Return the order_id and a display column that shows Unassigned when sales_rep is NULL. Then separately query only the truly NULL row.

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents

DE02-07 - Arithmetic expressions and safe calculations

What you are learning

Create row-level calculations safely and understand calculation order.

Why this matters

Revenue, discounts, margin and rates are often calculated rather than stored. One missing parenthesis can produce a plausible but wrong number.

Picture it in your head

SQL arithmetic follows normal mathematics: multiplication/division before addition/subtraction unless parentheses change the order. Parentheses also make your intent obvious to reviewers.

Start from zero

gross_sales = qty * unit_price.

If discount_pct is a decimal such as 0.10, net_sales can be gross_sales * (1 - discount_pct).

Use parentheses to make the discount logic explicit.

Validate calculated values manually for at least two rows before trusting a large result.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
SELECT order_id,
       qty * unit_price AS gross_sales,
       qty * unit_price * (1 - discount_pct) AS net_sales
FROM sales_orders
ORDER BY order_id
LIMIT 5;
```

Read the query in plain English

- For O1001: 2 x 850 = 1700 gross; discount is 0, so net is 1700.
- For O1002: 3 x 450 = 1350 gross; 5% discount means 1350 x 0.95 = 1282.50 net.

What success looks like

order_id	gross_sales	net_sales
O1001	1700.00	1700.00
O1002	1350.00	1282.50

Common beginner mistakes

- Subtracting discount_pct directly from money: 1350 - 0.05.
- Forgetting that 5% is represented as 0.05 in this dataset.
- Trusting the whole result without checking two known rows.

Now you do it

Calculate net_sales for all rows. Manually verify O1004 and O1020 with calculator arithmetic and write the two calculations as SQL comments.

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents

DE02-08 - Aggregate functions

What you are learning

Summarize many rows into one result using COUNT, SUM, AVG, MIN and MAX.

Why this matters

Business questions often ask "how many", "how much", or "what is the average". Aggregate functions compress a set of rows into summary values.

Picture it in your head

If every order is one receipt, an aggregate is the end-of-day summary: total receipts, total units, average price, smallest price and largest price.

Start from zero

COUNT(*) counts rows.

SUM(qty) adds quantities.

AVG(unit_price) averages the unit_price values across rows - not weighted by qty.

The meaning of an aggregate depends on what you aggregate. COUNT(order_id) and COUNT(DISTINCT customer_id) answer different questions.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
SELECT COUNT(*) AS order_rows,
       SUM(qty) AS total_qty,
       AVG(unit_price) AS avg_unit_price,
       SUM(qty * unit_price) AS total_gross_sales
FROM sales_orders;
```

Read the query in plain English

- This query has no GROUP BY, so it returns one summary row for the entire table.
- In the supplied data: 30 rows, total qty 105, total gross sales 50,000.

What success looks like

order_rows	total_qty	avg_unit_price	total_gross_sales
30	105	666.33 approx	50000.00

Common beginner mistakes

- Calling COUNT(*) "number of customers" when one customer could have multiple rows.
- Using AVG(unit_price) when the business asks for average selling price weighted by quantity.

Now you do it

For Completed orders only, return order count and total net_sales. Validate the row count separately with a second COUNT(*) query.

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents

DE02-09 - GROUP BY and grouped grain

What you are learning

Create one summary row per group and state the new result grain clearly.

Why this matters

GROUP BY changes the meaning of a result. Instead of one row per order, you may now have one row per region, category or payment method.

Picture it in your head

Imagine sorting 30 receipts into four region boxes, then calculating totals inside each box. Each box becomes one output row.

Start from zero

Columns in GROUP BY define the groups/buckets.

Aggregates such as COUNT and SUM calculate inside each bucket.

State the result grain before running the query: for example, one row per region.

Reconcile grouped totals back to the overall total whenever possible.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
SELECT region,
       COUNT(*) AS order_count,
       SUM(qty * unit_price) AS gross_sales
FROM sales_orders
WHERE status = 'Completed'
GROUP BY region
ORDER BY region;
```

Read the query in plain English

- WHERE first keeps only Completed rows.
- GROUP BY region then creates four regional buckets.
- COUNT and SUM are calculated separately inside each region.

What success looks like

region	order_count	gross_sales
East	8	12800
North	5	9310
South	7	12010
West	7	10930

Common beginner mistakes

- Forgetting that the grain changed from one row per order to one row per region.
- Selecting a non-aggregated column that is not in GROUP BY.
- Failing to reconcile grouped totals with the overall total.

Now you do it

Summarize Completed orders by payment_method with order_count, total_qty and gross_sales. Sum the group gross_sales values and compare them with a separate overall Completed gross-sales query.

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents

DE02-10 - HAVING after aggregation

What you are learning

Filter grouped results with HAVING and keep WHERE for row-level filtering.

Why this matters

WHERE and HAVING act at different stages. Mixing them up is one of the most common SQL-foundation mistakes.

Picture it in your head

WHERE decides which receipts are allowed into the boxes. GROUP BY creates the boxes. HAVING decides which completed boxes are large enough to show.

Start from zero

WHERE filters source rows before grouping.

HAVING filters groups after aggregates have been calculated.

Conditions such as status='Completed' belong in WHERE because status exists on individual rows.

Conditions such as SUM(qty) >= 25 belong in HAVING because the sum exists only after grouping.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
SELECT region, SUM(qty) AS total_qty
FROM sales_orders
GROUP BY region
HAVING SUM(qty) >= 25
ORDER BY region;
```

Read the query in plain English

- All rows are grouped by region.
- Only regional groups with total quantity at least 25 survive HAVING.

What success looks like

region	total_qty
East	29
North	27
West	25

Common beginner mistakes

- WHERE SUM(qty) >= 25.
- Putting every condition into HAVING just because the query has GROUP BY.
- Forgetting to explain whether the threshold applies to rows or groups.

Now you do it

For Completed orders, summarize by payment_method and keep only methods whose total quantity is at least 25. In the supplied data, Cash and UPI should remain.

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents

DE02-11 - CASE for business categories

What you are learning

Turn business rules into readable categories with CASE.

Why this matters

Companies often need labels such as High/Medium/Low, SLA breach bands, customer segments or risk categories. CASE lets SQL produce those labels without editing the source table.

Picture it in your head

CASE is a decision ladder. MySQL checks conditions from top to bottom and uses the first matching result.

Start from zero

Start with **CASE**, add one or more **WHEN condition THEN label** branches, add an **ELSE**, and finish with **END**.

Order overlapping conditions from most specific/highest threshold to more general thresholds.

ELSE makes unmatched rows visible instead of silently producing NULL.

The CASE expression can be aliased like any calculated column.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
SELECT order_id,
       qty * unit_price AS gross_sales,
       CASE
         WHEN qty * unit_price >= 2500 THEN 'High'
         WHEN qty * unit_price >= 1500 THEN 'Medium'
         ELSE 'Small'
       END AS order_size
FROM sales_orders;
```

Read the query in plain English

- An order of 2700 matches High immediately.
- An order of 1700 fails High but matches Medium.
- Everything below 1500 falls into Small.

What success looks like

bucket	row_count
High	3
Medium	14
Small	13

Common beginner mistakes

- Checking >=1500 before >=2500, which would make High unreachable.
- Leaving out ELSE and forgetting to investigate NULL categories.

Now you do it

Create net_sales first, then classify net_sales as High >= 2200, Medium >= 1200, else Low. Count the rows in each bucket as a separate validation query.

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents

DE02-12 - String and date basics

What you are learning

Clean simple text in query output and filter/extract basic dates without treating dates as arbitrary strings.

Why this matters

Real data contains spaces, inconsistent case and dates that need time-window analysis. You need a small toolkit before more advanced cleaning later in the course.

Picture it in your head

Text functions are like wiping labels before comparing them. Date functions are like reading the year/month/day fields from a proper calendar value instead of slicing random characters.

Start from zero

TRIM(text) removes leading/trailing spaces from the output.

UPPER and **LOWER** standardize case for display/comparison.

MySQL date values can be filtered with normal comparisons and BETWEEN.

Functions such as YEAR(order_date) and MONTH(order_date) extract calendar components in MySQL.

Do not convert every date to text just because text functions feel familiar.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
SELECT order_id,
       UPPER(TRIM(region)) AS clean_region,
       YEAR(order_date) AS order_year,
       MONTH(order_date) AS order_month
FROM sales_orders
WHERE order_date BETWEEN '2026-07-10' AND '2026-07-12';
```

Read the query in plain English

- BETWEEN is inclusive here: both 2026-07-10 and 2026-07-12 are included.
- The supplied data has 6 rows in that date range.

What success looks like

validation	expected
Rows in 10-12 July inclusive	6
Distinct cleaned region labels	4

Common beginner mistakes

- Using string slicing on a DATE column when a date-aware function exists.
- Assuming BETWEEN excludes the endpoints.
- Cleaning text in the result and then believing the stored source has been fixed.

Now you do it

Return order_id, a lowercase payment_method, order year and order month for the final five calendar days in the supplied table.

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents

DE02-13 - Simple subqueries and IN/EXISTS awareness

What you are learning

Understand a simple subquery and recognize IN/EXISTS as membership patterns without trying to master advanced SQL yet.

Why this matters

Sometimes one query needs the result of another query as an input. Subqueries let you nest a smaller question inside a larger one.

Picture it in your head

First ask: "What is the average order value?" Then ask: "Which orders are above that average?" The inner question supplies a number to the outer question.

Start from zero

A **scalar subquery** returns one value and can be used where one value is expected.

IN asks whether a value belongs to a returned list/set.

EXISTS asks whether a matching row exists. Its full power becomes clearer when we learn correlated subqueries later.

For foundations, focus on reading the inner query by itself first, validating it, then reading the outer query.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
SELECT order_id, qty * unit_price AS gross_sales
FROM sales_orders
WHERE qty * unit_price > (
    SELECT AVG(qty * unit_price)
    FROM sales_orders
)
ORDER BY gross_sales DESC;
```

Read the query in plain English

- Run the inner AVG query alone first. It returns one average value.
- The outer query then keeps only orders whose gross_sales is greater than that value.
- In the supplied data, 16 orders are above the average gross-sales row value.

What success looks like

validation	expected
Rows above average gross_sales	16

Common beginner mistakes

- Reading the whole nested query at once and getting lost.
- Using a multi-row subquery where the outer comparison expects one value.
- Skipping validation of the inner query.

Now you do it

Return Completed orders whose qty is greater than the average qty of all orders. Validate the inner average separately before running the outer query.

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents

DE02-14 - Debugging and validation workflow

What you are learning

Debug SQL systematically and validate correct-looking results before trusting them.

Why this matters

In real work, the dangerous query is not always the one that throws an error. A query can run successfully and still answer the wrong business question.

Picture it in your head

Treat SQL like a chain of reasoning. When the final answer is wrong, find the earliest link where your expectation and the evidence diverge.

Start from zero

First separate **syntax errors** (query will not run) from **logic errors** (query runs but answers the wrong question).

Read MySQL error messages and locate the first clause around the reported position.

Build complex queries in layers: source -> filter -> calculation -> grouping -> group filter -> sort/limit.

At every layer ask: what is the grain now, how many rows should I roughly expect, and what independent check can confirm it?

Use reconciliation: grouped totals should often add back to an overall total; filtered counts should match a separate COUNT query.

Teacher demonstration

Type this in a new MySQL Workbench query tab. Select the statement if needed, then click the lightning-bolt Execute button.

```
-- Broken on purpose
SELECT region, SUM(qty * unit_price) AS gross_sales
FROM sales_orders
WHERE SUM(qty) >= 25
GROUP BY region;
-- Question to ask:
-- Is SUM(qty) a row-level condition or a group-level condition?
```

Read the query in plain English

- SUM(qty) exists only after groups are formed, so this condition belongs in HAVING, not WHERE.
- The important skill is not memorizing the fix; it is identifying the stage where the value exists.

What success looks like

debug question	answer
Does the query parse/run?	Maybe not, depending on misuse
Earliest logical problem	Aggregate condition placed before grouping
Correct stage	HAVING after GROUP BY

Common beginner mistakes

- Changing several clauses at once and not knowing which fix mattered.
- Adding DISTINCT or LIMIT to make a result look reasonable.
- Accepting a result because it has no SQL error.

Now you do it

Take one previous query and create two independent validations for it: one row-count check and one reconciliation/spot-check. Save both below the main query as comments.

Save: your SQL + one validation note + confidence 0-5 + biggest uncertainty.

Before you mark this lesson mastered

Without reading the page, explain what each clause does, what one output row represents, and one way your result could look reasonable but still be wrong.

Back to contents